

Tri par Selection

0.1 Principe

Le tri par sélection consiste à parcourir successivement les positions du tableau et, pour chaque position i , à rechercher le plus petit élément parmi les éléments restants $t[i], \dots, t[n-1]$. Cet élément minimum est ensuite échangé avec l'élément situé en position i . Après la i -ième itération, les i premiers éléments du tableau sont donc définitivement triés et placés à leur position finale. Autrement dit :

- les $i - 1$ premiers éléments sont supposés triés,
- on recherche le minimum des éléments non triés i.e. parmi $t[i], \dots, t[n-1]$,
- on échange les emplacements dans le tableau du minimum et de $t[i]$

0.2 Complexité

Cas	Complexité
Meilleur cas (déjà trié)	$\Theta(n^2)$
Pire cas (ordre inverse)	$\Theta(n^2)$
Cas moyen	$\Theta(n^2)$

Remarque Le nombre de comparaisons ne dépend pas de l'ordre initial du tableau.

0.3 Invariant

$$I_{\text{int}} : \begin{cases} k \in \llbracket i+1, n+1 \rrbracket, \\ j \in \llbracket i, n \rrbracket, \\ t[j] = \min\{t[\ell] \mid i \leq \ell \leq k-1\}. \end{cases}$$

Et l'invariant de la boucle externe :

$$I_{\text{ext}} : \begin{cases} t \text{ est une permutation de } t_0, \\ i \in \llbracket 1, n+1 \rrbracket, \\ \forall a \in \llbracket 1, i-2 \rrbracket, t[a] \leq t[a+1], \\ \forall a \in \llbracket 1, i-1 \rrbracket, \forall b \in \llbracket i, n \rrbracket, t[a] \leq t[b]. \end{cases}$$

0.4 Pseudo-code (itératif)

Tri par Selection

Tri_selection (t, n)

Entrée : tableau t d'entiers de taille n

Sortie : tableau t, contenant les mêmes entiers qu'en entrée, trié par ordre croissant

// t[1] -> premier élément du tableau

```
i := 1
tant que i <= n faire :
    j := i
    k := i+1
    tant que k < n :
        si t[k] < t[j] :
            j := k
        finsi
    k := k+1
    fintantque
    t := Echanger (t, i, j)
    i := i+1
fintantque
```

Avec :

Echanger (t, i, j)

```
    tempo := t[j]
    t[j] := t[i]
    t[i] := tempo
retourner t
```

1 Implémentation en C

Tri par sélection

```
void tri_selection2(int *tab, int n) {
    int i = 0;
    int j, k;

    while (i < n) {
        min = i;
        j = i + 1;
        // Recherche du minimum
        while (j < n) {
            if (tab[j] < tab[min]) {
                min = j;
            }
            j++;
        }
        // Echange du minimum de ce qui reste et de l'indice où on est
        echanger(tab, i, min);
        i++;
    }
}
```

On reconnaît ici le principe du tri par insertion en deux étapes ...

Où l'on a utilisé la fonction basique :

Echanger

```
void echanger(int *t, int i, int j) {
    int tmp = t[i];
    t[i] = t[j];
    t[j] = tmp;
}
```

Une autre implémentation est possible en utilisant des boucles **for** :

Tri par sélection

```
void tri_selection(int *tab, int len) {
    int min;

    for (int i = 0; i < len; i++) {
        min = i;
        // Recherche du minimum
        for (int j = i + 1; j < len; j++) {
            if (tab[j] < tab[min]) {
                min = j;
            }
        }
        // Echange du minimum de ce qui reste et de l'indice où on est
        echanger(tab, i, min);
    }
}
```

Ce qui revient strictement au même.